IITM PRAVARTAK
CATALYSING INNOVATION

Advanced Certificate Programme in Agentic AI & RAG Engineering

30-WEEK PROGRAM

PHASE 1 — FOUNDATIONS

Python for AI Engineering

DAY 1

Typed contracts and async basics

Concepts → Pydantic + async fundamentals. Hands-on → three Pydantic demos + a tiny async script.

.

Welcome back — Week 2

First real-code week. Let’s check in

Who shipped

src/hello_llm.py

Who shipped

ADR v1 (canvas)

Anything stuck

on setup?

Today is the first real-code week.

- Async is genuinely new for some learners — that’s expected.
- The session is theory-light and practice-heavy; one 40-minute live build is the centre of gravity.
- Every learner should leave with a working `pipeline.py` on their machine.

Today — Day 1

Three hours together — typed contracts and async basics

0:00	Welcome back + W1 check-in
0:15	Why this week matters
0:25	Pydantic — concepts + three live demos
1:15	Break
1:30	Async fundamentals — sync vs async, mental model, tiny example
2:20	Sandbox / pair practice / Q&A
2:45	Day 1 wrap + preview Day 2

Why This Week Matters

The engineering posture the rest of the programme rests on

Today's async batch pipeline is the same shape we'll reuse from W6 onward.

- We don't touch capstone documents yet — we build the engineering foundation first.
- Pydantic + async + retry + logs is the minimum production posture for every later week.
- If async clicks this week, the rest of the programme is easier.

W8
Document-ingestion skeleton

W9
Parallel-query pattern

W11
Batch retrieval at scale

Pydantic for Structured Data

Validating data — the engineering antidote to flaky LLM output.

Why Pydantic

LLM outputs are unreliable. This is our first line of defence.

Validate inputs

Typed Python classes that check request payloads and config at runtime.

Validate outputs

Parse LLM responses into a known shape — and fail loudly when they don't match.

Clean errors

When something's wrong, you know exactly which field broke and why.

A Basic Pydantic Model

BaseModel — typed fields, sensible defaults

```python
from pydantic import BaseModel
from typing import Optional

class User(BaseModel):
    name: str
    age: int
    email: Optional[str] = None

u = User(name="Asha", age=29)
u.model_dump()
# {"name": "Asha", "age": 29, "email": None}
```

When Bad Data Goes In

One clear error, exactly where it broke

```
# Wrong type for age
>>> User(name="Asha", age="not a number")

pydantic.ValidationError: 1 validation error for User
  age
    Input should be a valid integer  [type=int_parsing]

# No silent corruption. No untyped dict floating through the system.
# You always know which field broke — and why.
```

The AI Use Case

Parse an LLM response into a known shape

```python
class ExpectedAnswer(BaseModel):
    summary: str
    confidence: float
    sources: list[str]

raw = '{"summary": "...", "confidence": 0.9, "sources": ["d1"]}'
parsed = ExpectedAnswer.model_validate_json(raw)

# parsed.summary    -> str, guaranteed
# parsed.confidence -> float, guaranteed
# parsed.sources    -> list[str], guaranteed
# If the LLM returns a malformed JSON or wrong shape:
#   ValidationError tells you exactly which field broke.
```

Break

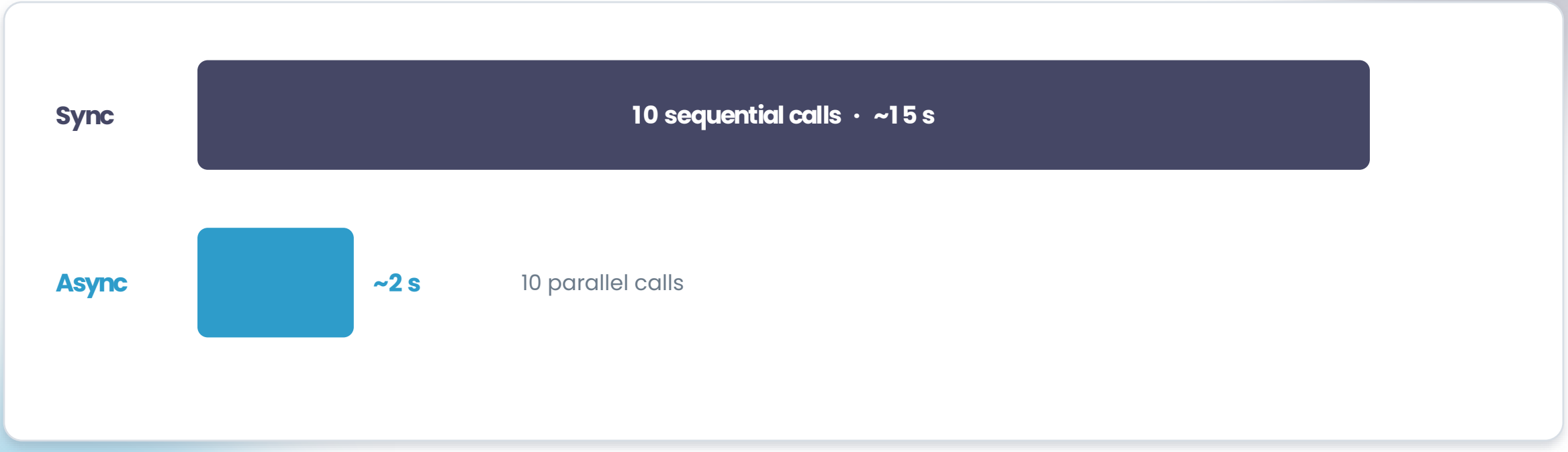
15 minutes — ADR and Canvas next..

Async Python

asyncio + httpx — the parallelism unlock.

Sync vs Async — Same 10 Calls

Why we async every I/O call from here on



Same calls. Same API. Same answer quality. ~10× faster — and that gap grows with N.

The Mental Model

Three keywords. One sentence to remember.

`async def`

Mark a function as awaitable. Doesn't run anything by itself.

`await`

Pause this coroutine until the awaited thing is done. Yield to the event loop while we wait.

`asyncio.run(…)`

The entry point. Start the event loop, run the top-level coroutine to completion.

When you await an I/O call, the event loop runs other tasks instead of blocking — that's how 10 calls finish in the time of one.

The Tiniest Async Example

async def + await + asyncio.run

```python
import asyncio

async def hello(name):
    await asyncio.sleep(0.1)
    return f"hi, {name}"

asyncio.run(hello("Asha"))
# 'hi, Asha'

# Same shape every async program has:
#   1. async def a coroutine.
#   2. await the I/O bits inside it.
#   3. asyncio.run() once at the top to start.
```

httpx — Our Async HTTP Client

Drop-in replacement for requests, with async

```python
import httpx

async def fetch(url):
    async with httpx.AsyncClient() as client:
        r = await client.get(url)
        return r.json()

# AsyncClient handles connection pooling and TLS reuse.
# Use it for every outbound HTTP call from now on.
# The OpenAI SDK already uses an async-capable client internally —
# we'll use AsyncOpenAI directly in the hands-on.
```

Day 1 — wrap

What we covered, what's next tomorrow

Today, you...

- Saw Pydantic validate three shapes — basic, bad input, AI response.
- Learned async/await — coroutines, the event loop, why it's faster for I/O.
- Watched a sync-vs-async timing comparison — ~10× speedup is real.

Tomorrow we...

- Stack three concurrency patterns — gather, batching, retry.
- Build the full async pipeline live, end to end — twice.
- Add the operational baseline — logging, secrets, SQLite.
- Walk through this week's lab.

DAY 2

Concurrency, the build, the baseline

Concepts → three concurrency patterns + operational baseline. Hands-on → the full pipeline, built live.

Welcome back

A quick check-in before we dive in

Did you try the
Pydantic demos yourself?

Run a quick async
sleep example?

Anything still confusing
from Day 1?

Today we put it all together.

- Yesterday: typed contracts (Pydantic) and the async/await mental model.
- Today: the three patterns that stack into a real pipeline — gather, batching, retry.
- By the end of today: you've built and run the W2 pipeline live, twice (clean + lossy).

Today — Day 2

Three hours together — concurrency, the build, the baseline

0:00	Welcome back + Day 1 recap
0:15	Three concurrency patterns — gather, batching, retry
1:00	Break
1:15	Live build — the full async pipeline
2:15	Operational baseline — logging, secrets, SQLite, AI assistant
2:45	Lab walkthrough + key takeaways + Q&A

Concurrency Patterns

Three patterns we'll use everywhere from here on.

asyncio.gather — fire N, collect all

PATTERN 1 OF 3

```python
import asyncio

async def call(i):
    await asyncio.sleep(1)
    return i * 2

results = await asyncio.gather(*(call(i) for i in range(10)))
# 10 calls fire together. Total time: ~1 s, not 10 s.
# gather waits for all of them and returns results in order.
```

Batching — don't fire 1,000 at once

PATTERN 2 OF 3

```python
async def batched(items, size, fn):
    out = []
    for i in range(0, len(items), size):
        chunk = items[i:i+size]
        out += await asyncio.gather(*(fn(x) for x in chunk))
        await asyncio.sleep(0.1)  # gentle pace
    return out

# Why: rate limits, memory, and your bank account.
# 1000 items at size=10 → 100 batches, each runs gather internally.
```

Retry with exponential backoff

PATTERN 3 OF 3

```python
async def with_retry(coro_fn, *args, tries=3):
    for attempt in range(tries):
        try:
            return await coro_fn(*args)
        except Exception:
            if attempt == tries - 1: raise
            await asyncio.sleep(2 ** attempt)  # 1 s, 2 s, 4 s

# Simple, readable, no library.
# When you outgrow this: tenacity (decorator-based retries).
```

All Three, Stacked

This is the shape of the pipeline we'll build next

1

`gather`

Fire a batch in parallel

2

`batched`

Group items into batches

3

`with_retry`

Try each call up to 3 times

Pydantic + gather + batched + with_retry + structured logs = the minimum production pipeline.

Break

15 minutes — ADR and Canvas next..

Build the Pipeline Together

Five steps. One working async batch pipeline. Type along.

Pydantic input & output models

STEP 1 OF 5

```python
# pipeline.py
from pydantic import BaseModel

class Question(BaseModel):
    text: str

class Answer(BaseModel):
    question: str
    text: str
    cost_usd: float
    retries: int = 0
```

async def ask_llm — one parallel-safe call

STEP 2 OF 5

```python
from openai import AsyncOpenAI
client = AsyncOpenAI()  # reads OPENAI_API_KEY from env

async def ask_llm(q: Question) -> Answer:
    resp = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": q.text}],
    )
    return Answer(
        question=q.text,
        text=resp.choices[0].message.content,
        cost_usd=0.0001,  # real cost calc lands in W25
    )
```

Wrap with retry + exponential backoff

STEP 3 OF 5

```python
import asyncio

async def ask_llm_with_retry(q: Question, tries: int = 3) -> Answer:
    for attempt in range(tries):
        try:
            ans = await ask_llm(q)
            ans.retries = attempt
            return ans
        except Exception:
            if attempt == tries - 1: raise
            await asyncio.sleep(2 ** attempt)  # 1 s, 2 s, 4 s
```

run_batch — gather it all together

STEP 4 OF 5

```python
async def run_batch(questions: list[Question]) -> list[Answer]:
    tasks = [ask_llm_with_retry(q) for q in questions]
    return await asyncio.gather(*tasks)

if __name__ == "__main__":
    sample = [Question(text=t) for t in [
        "What is RAG in one sentence?",
        "Name three uses of vector databases.",
        "Why might an LLM hallucinate?",
    ]]
    answers = asyncio.run(run_batch(sample))
    for a in answers: print(a.text[:80])
```

Structured (JSON) logging

STEP 5 OF 5

```python
import logging, json, time

class JsonFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({
            "ts": round(time.time(), 3),
            "level": record.levelname,
            "msg": record.getMessage(),
        })

log = logging.getLogger("pipeline")
log.setLevel(logging.INFO)
h = logging.StreamHandler(); h.setFormatter(JsonFormatter())
log.addHandler(h)

# inside ask_llm, after the API call:
log.info(f"asked: {q.text[:40]}")
```

Run It — What We See

Three questions, in parallel, with one retry

```
$ python -m src.pipeline

{"ts": 1717.034, "level": "INFO", "msg": "asked: What is RAG in one sentence..."}
{"ts": 1717.041, "level": "INFO", "msg": "asked: Name three uses of vector data..."}
{"ts": 1717.043, "level": "WARN", "msg": "retry 1 for: Why might an LLM hallucina..."}
{"ts": 1717.892, "level": "INFO", "msg": "asked: Why might an LLM hallucinate?"}
{"ts": 1717.945, "level": "INFO", "msg": "done: 3 answers in 1.91s"}

RAG combines retrieval over a document corpus with...
Vector databases power semantic search, RAG context...
LLMs can produce confident but unfounded text when...

# 3 questions · 1.91 s in parallel (sync would be ~5.5 s)
```

Operational Baseline

Logs, secrets, persistence — and a habit for AI coding assistants.

Why Structured Logging

One JSON line per call — and everything follows

Machine-readable

Every log is JSON. Grep, jq, and dashboards just work — no regex acrobatics.

Per-call traceability

Each LLM call becomes one searchable record: prompt, latency, retries, cost.

Foundations for KPIs

From W6, this same log feeds hallucination rate, retrieval hit rate, cost per query.

Secrets — Extending the W1 Discipline

.env + python-dotenv + .gitignore

.env

Local file. One key per line. Never committed.

python-dotenv

load_dotenv() reads .env into os.environ before any client is created.

.gitignore

Lists .env. Run git ls-files | grep -q '^\.env$' to verify.

```
# .env (gitignored)
OPENAI_API_KEY=sk-...

# anywhere in your code
from dotenv import load_dotenv
load_dotenv()

# clients pick it up automatically
from openai import AsyncOpenAI
client = AsyncOpenAI()
```

SQLite in Five Lines

The simplest possible local store — enough for now

```python
import sqlite3

con = sqlite3.connect("results.db")
con.execute("CREATE TABLE IF NOT EXISTS answers ("
            "id INTEGER PRIMARY KEY, q TEXT, a TEXT, cost_usd REAL)")
con.execute("INSERT INTO answers (q, a, cost_usd) VALUES (?, ?, ?)",
            ("What is RAG?", "...", 0.0001))
con.commit()

# Built into Python — no server, no setup, just a file on disk.
# Use it when you need to persist a few thousand rows reliably.
# Vector databases come in W7. Postgres if/when needed later.
```

AI Coding Assistant — Habit, Not Tool

Cursor, Copilot, Claude Code — the same five steps

1

Ask clearly

Describe the change. The clearer the ask, the smaller the diff.

2

Read the diff

Don't skim. Every line you accept becomes yours to defend.

3

Run the test

If there's no test, write one before merging anything non-trivial.

4

Check for security

Hard-coded keys, unsanitised input, new dependencies — eyeball them.

5

Accept

Commit with a clear message. The tool drafted it; you shipped it.

The deliverable in this week's lab is the verification workflow itself — not a polished feature.

This Week's Lab

≈ 3 hours · four steps · all on Vocareum

1 **Project skeleton + logging config** — ~30 min

Create src/pipeline/, add deps, set up JSON logging to logs/pipeline.log. Confirm .env is gitignored.

2 **Extend the async batch pipeline** — ~90 min

Read 20 questions from data/questions.csv. Run with batches of 5, retries, structured logs. Write results.json.

3 **Add SQLite persistence** — ~45 min

Replace JSON output with results.db. Write query_results.py to search answers by pattern.

4 **Coding-assistant mini-exercise** — ~20 min

Make one small improvement using your assistant. Document the verification workflow in docs/lab2-assistant-notes.md.

Starter pipeline.py with imports and TODOs is in the cohort repo — start there.

Key Takeaways

What to carry into Week 3

Pydantic validates

Define every input, output, and LLM response as a Pydantic model.

Async unlocks parallel

async def + await + asyncio.run. Use AsyncOpenAI and httpx.

Three patterns stack

gather + batched + with_retry — the minimum production loop.

One JSON log per call

Structured logs feed every KPI we'll define from W6 onward.

If async clicked, the rest of the programme is easier. If it didn't fully click — the lab is where it lands.

Q & A

Thank you